Board Guide

Building and Measuring Every Rung of the Improvement Ladder on the ZCU102

TeraNova – FPGA Acceleration

Author	Kai Janssen (4994817)
Version	30 October 2026
Board	AMD ZCU102 (Zynq UltraScale+ `xczu9eg-ffvb1156-2-e`)
Tools	Vivado / Vitis 2025.1.1, git, PuTTY
Repository	`github.com/GiganticGnu/TeraNova-FPGA`, at `C:\PLD`

Board details in this guide come from the ZCU102 Evaluation Board User Guide (UG1182) and the Zynq UltraScale+ MPSoC Embedded Design Tutorial (UG1209); see the references at the end.

Contents

1 What this guide is for **2**

2 One-time preparation **2**

2.1 Software on the PC that drives the board 2

2.2 The repository 3

3 The board **3**

3.1 The parts you use 3

3.2 Connecting and switching on 4

3.3 The serial log (PuTTY) 4

4 Phase A – Build a rung (licensed PC) **5**

5 Phase B – Prepare Vitis for a rung (no licence) **6**

6 Phase C – Run and measure **7**

7 What you should see, rung by rung **8**

7.1 Output of the baseline application (rungs 0–3) 8

7.2 Output of the burst application (rung 4) 8

7.3 Expected values 9

8 Result sheet **9**

9 Troubleshooting **9**

10 Board-day checklist **10**

References **11**

1 What this guide is for

The accelerator is being improved step by step. Every step is one *rung* of the improvement ladder, and every rung is stored as its own git branch. Each rung contains all earlier rungs plus one new change. To see what each change brings, the rungs are measured on the board **one at a time, in this order**:

Rung	Git ref	What changes	Application	PL time (sim.)
0	tag `baseline-1` (`main`)	Nothing: the restored design, the reference	baseline	1012 µs
1	`partitioner-v5b`	Partitioner fetches one element per cycle	baseline	272 µs
2	`writer-fix`	Result writer no longer waits for every write response	baseline	271 µs
3	`partitioner-v6`	Partitioner copies A and B on chip once, reuses them 10×	baseline	101 µs
4	`burst-datapath`	PL reads A/B from and writes C to the processor's DDR; no BRAMs	burst	101 µs

Table 1: The five rungs. "PL time" is the simulated start-to-`done` time at 100 MHz for the nominal memory latency (Section 7 gives the full expected range). The application is already the right one on every branch.

For every rung the same three phases are done (Figure 1). Phase A needs the Vivado licence, so it runs on Badr's laptop. Phases B and C need no licence and can be done on any PC that has the tools and is connected to the board.

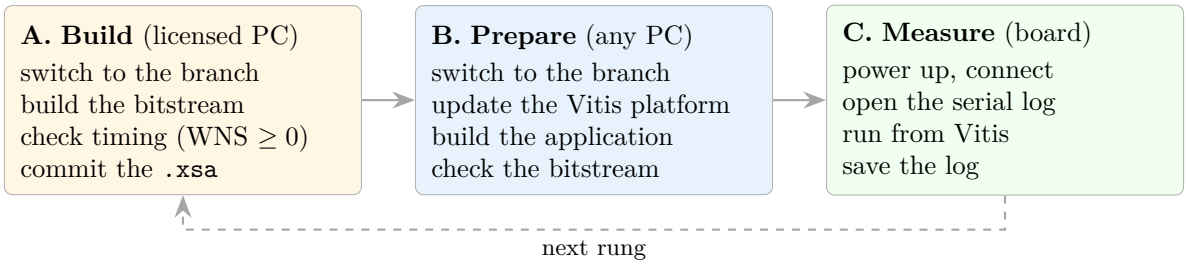


Figure 1: The three phases, repeated for every rung.

Important. Never mix rungs. The hardware file (`.xsa`), the Vitis platform and the application must all come from the *same* branch. A run with a bitstream of one rung and the platform of another either fails or, worse, gives believable but wrong numbers. Every phase below starts with a check of the branch for exactly that reason.

2 One-time preparation

2.1 Software on the PC that drives the board

1. **Vivado and Vitis 2025.1.1**, installed in `C:\Xilinx\2025.1.1` with Zynq UltraScale+ MPSoC support. Only this version: 2025.1 locks every IP of the project.

2. **JTAG cable drivers.** Run once, as administrator:

```
C:\Xilinx\2025.1.1\data\xicom\cable_drivers\nt64\install_drivers_wrapper.bat
```

3. **USB-UART driver.** The board's serial port is a Silicon Labs CP2108 quad USB-UART bridge. Install the *CP210x Universal Windows Driver* (VCP driver) from the Silicon Labs website if Windows does not install it by itself.

4. **git** (Git for Windows, includes Git Bash) and **PuTTY** (serial terminal that can write a log file).

2.2 The repository

The project must be at exactly `C:\PLD`: the Vivado project and the Vitis workspace contain absolute paths. If `C:\PLD` does not exist yet, open Git Bash and run:

```
git clone https://github.com/GiganticGnu/TeraNova-FPGA.git /c/PLD
```

If it already exists, bring it up to date:

```
cd /c/PLD
git status          # must say "nothing to commit, working tree clean"
git fetch --all --tags
```

Finally, make a folder for the results, outside the repository: `C:\PLD_out\`. Every rung gets its own sub-folder, named after the rung: `0_baseline`, `1_partitioner_v5b`, `2_writer_fix`, `3_partitioner_v6`, `4_burst_datapath`.

3 The board

3.1 The parts you use

Figure 2-1 of UG1182 shows where everything is; the reference designators are also printed on the board itself.

Part	What it is	What you do with it
`J52`	12 V power connector	Plug in the board's own 12 V supply (6-pin connector). Use no other supply.
`SW1`	Power on/off slide switch	Switch the board on and off. Always switch off before you change cables or switches.
`SW6`	4-pole DIP switch: boot mode	Must be set to **JTAG**: all four switches **ON** (mode pins 0000). This is also the factory default. In JTAG mode the processor waits for Vitis to load the program; it does not boot from the SD card or flash.
`J2`	Micro-USB: JTAG (on-board Digilent USB-to-JTAG module)	Cable 1 to the PC. Vitis uses it to program the FPGA and load the program.
`J83`	Micro-USB: USB-UART (CP2108, 4 channels)	Cable 2 to the PC. Channels 0 and 1 are the processor's UARTs, channel 2 goes to the PL, channel 3 to the board's system controller. Our program prints on **channel 0** (PS UART0).

Part	What it is	What you do with it
DS32	Green LED "DONE"	Lights when the FPGA has been programmed ("bit file download is complete").
Green power LEDs	e.g. DS21–DS26, DS19, DS31	Light when the supply rails are on.
DS35, DS36	Red LEDs: PS error, power-on reset	Must be **off** while the board runs.
SW4, SW3	Push buttons: power-on reset (POR_B), system reset (SRST)	Not needed for this guide; use SW1 to restart the board instead.

Important. The board is sensitive to static electricity. Touch a grounded metal object before you handle it, hold it by the edges, and keep it on its standoffs on a non-conductive surface. Never connect or disconnect anything while SW1 is on.

3.2 Connecting and switching on

Do this once at the start of a board session.

1. With SW1 **off**: check that SW6 is set to JTAG (all four ON).
2. Connect the 12 V supply to J52.
3. Connect micro-USB cable 1 from J2 (JTAG) to the PC, and cable 2 from J83 (UART) to the PC.
4. Switch SW1 on. The fan starts, the green power LEDs light, and the red LEDs DS35/DS36 stay off. DS32 (DONE) is still off: no bitstream has been loaded yet.
5. **Find the serial port.** Open the Windows Device Manager → Ports (COM & LPT). You see four entries "Silicon Labs Quad CP2108 USB to UART Bridge: Interface 0" to "Interface 3", each with a COM number. **Interface 0 is PS UART0**, where the program prints. Write down its COM number (UG1209: "MPSoC UART-0 corresponds to the COM port with Interface-0"). If you see no CP2108 entries, the driver is missing (Section 2).
6. **Check the JTAG connection** (only needed the first time, or when something fails). In Vivado: Open Hardware Manager → Open target → Auto Connect. The hardware tree must show the xczu9_0 device. Then **close the Hardware Manager**: as long as it is open it holds the JTAG cable and Vitis cannot use it.

3.3 The serial log (PuTTY)

Everything the program prints is the measurement, so it is written to a file directly.

1. Start PuTTY. Session: Connection type Serial, Serial line COMx (the Interface 0 port), Speed 115200.
2. Connection → Serial: 8 data bits, 1 stop bit, parity None, flow control None.
3. Session → Logging: select Printable output; Log file name C:\PLD_out\<rung>\board_log.txt, for example C:\PLD_out\0_baseline\board_log.txt; select Ask the user for an existing file.

4. Click Open. The window stays empty until the program runs.

Save these settings under a name (Session → Saved Sessions, e.g. ZCU102); then only the log file name has to be changed per rung. The Vitis Serial Monitor works too, but it does not write a log file.

4 Phase A – Build a rung (licensed PC)

This phase turns the design of one branch into a hardware file, `interrupt_rerun.xsa`, which contains the bitstream. It is the only phase that needs the licence. Do the rungs in ladder order.

1. **Start clean.** Close Vivado and Vitis. In Git Bash:

```
cd /c/PLD
git status                # must be clean; if not, see the note below
git fetch --all --tags
```

2. **Switch to the rung.**

```
git switch main               # rung 0  (main = tag baseline-1)
git switch partitioner-v5b    # rung 1
git switch writer-fix         # rung 2
git switch partitioner-v6     # rung 3
git switch burst-datapath     # rung 4
git pull                      # then: get the latest commit of that branch
git log -1 --oneline          # check the commit (table below)
```

   Only the line of the rung you build, of course. **Expected commit** (`git log -1` shows this one, or a later "Build of rung" commit on top of it):

Rung	Branch	Commit	Message starts with
0	`main`	`3c52095`	Measurement app, simulation testbenches
1	`partitioner-v5b`	`5c49045`	Partitioner v5b (pipelined fetch)
2	`writer-fix`	`70ba245`	Writer fix: explicit bursts
3	`partitioner-v6`	`b056560`	Partitioner v6: on-chip copy
4	`burst-datapath`	`b99091c`	Rung 4: burst data path

3. **Open the project.** Start Vivado **2025.1.1** and open `C:\PLD\S7_integration\S7_integration.xpr`. The many `[Board 49-26]` warnings and the "generated file not found ... regenerate" warnings are normal. Reports → Report IP Status: nothing may be locked. (On rung 4 the partitioner is `partitioner_v7_0`, an RTL module; that is correct.)

4. **Build.** In the Tcl Console:

```
source C:/PLD/scripts/3_build_and_export.tcl
```

   This takes a long time (synthesis, implementation, bitstream). At the end it prints `Worst negative slack: ... ns` and writes `C:\PLD\S7_integration\interrupt_rerun.xsa`.

5. **Check timing.** The worst negative slack (WNS) must be ≥ 0. If it is negative, timing is not met: stop here and report it. Do not run this rung on the board.

6. **Save the reports.** In the Tcl Console (use the folder of this rung):

```
file mkdir C:/PLD_out/0_baseline
open_run impl_1
report_utilization    -file C:/PLD_out/0_baseline/utilization.txt
report_timing_summary -file C:/PLD_out/0_baseline/timing.txt
file copy -force C:/PLD/S7_integration/interrupt_rerun.xsa C:/PLD_out/0_baseline/
```

7. **Close Vivado**, then commit only the hardware file, and throw away everything else the build changed:

```
git add S7_integration/interrupt_rerun.xsa
git commit -m "Build of rung 0 (WNS 0.123 ns)"
git push
git restore .
git status                # must be clean again
```

Put the real rung number and WNS in the message.

Why. The `.xsa` is tracked by git. Committing it to the rung's own branch makes the build available to everyone: phases B and C can then be done on any PC, without the licence and without copying files by hand. The other files the build touches (the project file, IP state) are not needed and are restored, so that the next `git switch` works.

Important. If `git status` is not clean before you start, or `git switch` says "Your local changes … would be overwritten", do *not* force anything. Changes left over from a build are thrown away with `git restore .`; anything else, ask first.

5 Phase B – Prepare Vitis for a rung (no licence)

1. **Switch to the rung and get its build.** Close Vitis. In Git Bash:

```
cd /c/PLD
git status                # clean
git switch writer-fix     # the branch of this rung (see Phase A)
git pull
git log -1 --oneline -- S7_integration/interrupt_rerun.xsa
```

The last command must show **"Build of rung** N**"** for *this* rung. If it shows `2e936e3 Baseline: restored working build`, this branch has not been built yet: the `.xsa` is still the previous group's file. Do not continue with that file.

2. **Check the application.** Open `C:\PLD\S7_integration_vit\interrupt_rerun_test\src\helloworld.c`. Line 3 must say `BASELINE MEASUREMENT VERSION` for rungs 0–3, or `LADDER STEP 4: BURST DATA PATH` for rung 4. (This is already so on every branch.)

3. **Open the workspace.** Start Vitis **2025.1.1** → Open Workspace → `C:\PLD\S7_integration_vit`.

4. **Update the platform.** In the Vitis Components view, open `interrupt_rerun_plat` → Settings → `vitis-comp.json`. Next to the hardware specification, choose Switch XSA (update hardware specification) and select `C:\PLD\S7_integration\interrupt_rerun.xsa`. Confirm. Then, with `interrupt_rerun_plat` selected in the Flow panel, click Build. It must end without errors.

> **Why.** The platform keeps its own copy of the hardware description and generates the drivers and `xparameters.h` from it. Without this step the application is built for the hardware of another rung. Rung 4, for example, has no BRAM controllers and a new HP0 port.

5. **Build the application.** Select `interrupt_rerun_test` in the Flow panel → Build. It must end with a size summary and no errors.

6. **Check the bitstream.** The run configuration programs the FPGA from `C:\PLD\S7_integration_vit\interrupt_rerun_test\_ide\bitstream\interrupt_rerun.bit`. Check in Explorer that this file has **today's date and time**. If it does not exist or is older, copy it from `C:\PLD\S7_integration_vit\interrupt_rerun_plat\export\interrupt_rerun_plat\hw\sdt\interrupt_rerun.bit` (create the `_ide\bitstream` folder if needed).

> **Why.** An old bitstream is the most likely way to measure the wrong rung without noticing it. The two-second date check prevents that.

6 Phase C – Run and measure

1. The board is on and connected (Section 3.2), the Vivado Hardware Manager is closed.
2. Open PuTTY with the log file of *this* rung (Section 3.3).
3. In Vitis, select `interrupt_rerun_test` in the Flow panel and click Run. Vitis now resets the board, programs the FPGA (`DS32` DONE lights up), initialises the processor and loads and starts the program.
4. The PuTTY window shows the output. A run takes a few seconds. It ends with `ALL RUNS PASSED` (or `SOME RUNS FAILED`).
5. **Run it a second time** (Run again) so that every rung has two independent sets of 10 runs. Both go into the same log.
6. Close PuTTY (this closes the log file). Check that `C:\PLD_out\<rung>\board_log.txt` contains both runs.
7. Fill in the result sheet (Section 8).
8. **Clean up before the next rung.** Close Vitis. In Git Bash:

```
cd /c/PLD
git restore .
git clean -fd S7_integration_vit/interrupt_rerun_plat
git status                # must be clean
```

   The platform update changed tracked files; they are thrown away here, so that the next `git switch` works and the next rung starts from its own files. Your logs in `C:\PLD_out\` are outside the repository and are not affected.

You can leave the board switched on between rungs; every Run resets it and loads a fresh bitstream. Switch it off with `SW1` at the end of the session, then unplug the cables.

7 What you should see, rung by rung

7.1 Output of the baseline application (rungs 0–3)

```
50x50 matrix multiplication on the PL - BASELINE MEASUREMENT
Build: optimised (-O1 or higher)
--- CPU-only 50x50 multiply (Cortex-A53) ---       CPU reference
--- Latency probes (average of 1000 accesses) ---  one BRAM write/read, one PL-DDR4
    read
CSV,probe,<write ns>,<read ns>,<ddr ns>
--- 10 runs on the PL (times in us) ---
Run  1: load ... compute ... read-back ... total ... us  PASSED
CSV,run,1,<load ns>,<compute ns>,<read-back ns>,<total ns>,<errors>,<polled>
...
--- Summary over passing runs (us) ---              min / avg / max per phase
Runs passed: 10 of 10
done detected by: interrupt 10, GPIO poll 0 (+-1 us), not seen 0
ALL RUNS PASSED
```

- **load**: the CPU writing A and B into the BRAMs (10 000 single writes);
- **compute**: start to `done` on the PL, the number the simulation predicts;
- **read-back**: the CPU reading the 2 500 results from the PL DDR4.

7.2 Output of the burst application (rung 4)

Rung 4 prints the same blocks without the latency probes, plus:

```
Buffers in PS DDR: A 0x800000000  B 0x800005000  C 0x80000a000
Buffer check: CPU can write, flush, invalidate and read A, B and C (ok)
Run  1: flush ... compute ... inval+read ... total ... us  PASSED
CSV,burst,1,<flush A+B ns>,<invalidate C ns>,<read C ns>
```

“load” is now the cache flush of A and B, and “read-back” is the cache invalidation plus the reading of C, in the same CSV columns as before.

7.3 Expected values

Rung	PL compute (sim.)	What should change compared with the rung before	Check in particular
0	1012–1016 µs	– (reference)	The probe line: these numbers replace the estimates in the ladder. `done detected by: interrupt 10`.
1	272–586 µs	compute much lower; load and read-back unchanged	Where compute lands in its range shows the PL-DDR4 write latency: 272 means ≤370 ns, higher means slower.
2	271–272 µs	compute equal to or lower than rung 1	Compute no longer depends on the memory latency.
3	101–181 µs	compute lower again; load and read-back unchanged	–
4	101–182 µs	compute about equal; load and read-back far lower	`Buffer check ... (ok)`; the `CSV,burst` lines.

Table 3: Expected compute time per rung. The lower value is the simulation at a memory latency of 20 cycles (200 ns), the upper at 100 cycles (1 µs). The measured compute time also contains the start signal and the interrupt, so it can be a microsecond or two higher. Load and read-back are not simulated; they are what the board measurement is for.

A compute time far outside its range means the hardware does not behave like the simulated RTL, most often because of a stale bitstream or a platform of another rung (Section 9).

8 Result sheet

Copy the averages from the `Summary` block of the first run of the program (all in µs). Keep the log files: all CSV lines are in there for later analysis.

Rung	Date	WNS (ns)	Load	Compute	Read-back	Total	CPU only	Pass / IRQ
0								
1								
2								
3								
4								

Rung 0 also: `CSV,probe` values (write / read / DDR read, ns): _____

Send the log files (or push them in a `results` commit on `main`) so that the ladder's CPU estimates can be replaced by these measurements.

9 Troubleshooting

Symptom	Cause and what to do
No green LEDs after `SW1` on	No power: check the 12 V supply at `J52` and the mains.
Red `DS35` or `DS36` lights	PS error or power-on reset active. Switch off with `SW1`, wait 10 s, switch on. If it stays on, stop and ask.
No CP2108 ports in Device Manager	Cable not in `J83`, board off, or the CP210x driver is missing.
PuTTY stays empty	Wrong COM port (use *Interface 0*), or the program did not start: look at the Vitis console for an error.
Unreadable characters in PuTTY	Speed is not 115 200, or wrong data format (must be 8N1, no flow control).
Vitis: no target / JTAG error	Cable not in `J2`, board off, `SW6` not on JTAG (all ON), or the Vivado Hardware Manager is still open. Cable drivers not installed (Section 2).
`DS32` (DONE) stays off after Run	The FPGA was not programmed: check that the run configuration has Program Device on and that the bitstream file of Phase B, step 6 exists.
`no done after ... us`	The PL did not finish. Most likely a bitstream or platform of another rung: repeat Phase B with the date check. On rung 4 the HP0 port must be in the hardware (rung-4 build).
`done detected by: interrupt 0, GPIO poll 10`	The PL works, but the interrupt does not arrive. The compute time is still valid to about 1 µs. Note it in the result sheet.
`FAILED` with mismatches	Wrong results. Note the first mismatches from the log. On rung 4 a cache problem is the first suspect; the start-up buffer check must say `ok`.
`Buffer check FAILED` (rung 4)	The CPU cannot use DDR high (`0x8_0000_0000`). Make sure the platform was built from the rung-4 `.xsa`.
Application build fails on `xbram.h` (rung 4)	`bram_write.c` is still in the application: check that you are on `burst-datapath` and that `helloworld.c` is the burst version.
WNS < 0 in Phase A	Timing not met at 100 MHz: do not measure this rung; report it with `timing.txt`.
Licence error in Phase A	This PC has no licence for the `xczu9eg`: build on the licensed PC.
All IPs locked when opening the project	Vivado 2025.1 instead of 2025.1.1. Close without saving, `git restore .`, start 2025.1.1.
`git switch` refuses (local changes)	Leftovers of a build or platform update: `git restore .` (and `git clean -fd S7_integration_vit/interrupt_rerun_plat`), then switch again.

10 Board-day checklist

Once per session

- ☐ `SW1` off, `SW6` on JTAG (all ON)
- ☐ 12 V at `J52`, USB to `J2` (JTAG) and `J83` (UART)
- ☐ `SW1` on: green LEDs on, red LEDs off
- ☐ COM port of "Interface 0" noted
- ☐ Vivado Hardware Manager closed

Per rung (in order 0, 1, 2, 3, 4)

- ☐ `git status` clean; `git switch <branch>`; `git pull`
- ☐ `git log -1 - S7_integration/interrupt_rerun.xsa` shows "Build of rung N"
- ☐ Vitis: platform Switch XSA → Build; application Build
- ☐ `_ide\bitstream\interrupt_rerun.bit` has today's date
- ☐ PuTTY open, logging to `C:\PLD_out\<rung>\board_log.txt`
- ☐ Run twice; `ALL RUNS PASSED`; compute time within Table 3
- ☐ Result sheet filled in
- ☐ Vitis closed; `git restore .`; `git clean -fd S7_integration_vit/interrupt_rerun_plat`

References

[1] AMD/Xilinx, *ZCU102 Evaluation Board User Guide* (UG1182): board components (Table 2-1), MPSoC device configuration and boot modes, CP2108 USB UART interface, power and status LEDs (Table 3-34). Pages consulted via https://www.manualslib.com/manual/1542907/Xilinx-Zcu102.html.

[2] AMD/Xilinx, *Zynq UltraScale+ MPSoC Embedded Design Tutorial* (UG1209), "Connecting the Serial Port": https://docs.amd.com/r/2025.2-English/ug1209-embedded-design-tutorial/Connecting-the-Serial-Port.

[3] K. Janssen, *Setup Guide* (`C:\PLD\report\setup_guide.tex`), and *Simulation Report*, Section "The improvement ladder".